

100 Days of Azure DevOps

Day 93 Handout — If a stranger cannot recreate the env from Git, the demo is still a toy

10 - Portfolio & Public Launch · Personal lab only · Educational content · Not a sales pitch

Architecture A — infra for the same public demo, two environments

Box	Personal Azure / Git object	Rule
Module / file	capstone/infra/ (Bicep or Terraform — same dialect as Phase 5)	Do not switch languages this week for spice
Dev parameters	dev.bicepparam or terraform/dev.tfvars	SKU F1 / consumption; short names via uniqueString
Staging parameters	stg.bicepparam or terraform/stg.tfvars	Same module, different param file — no infra-prod-FINAL folder
State / deploy	Remote state (Day 48) or a pipeline service connection	Local state is a rumor you cannot share
Pipeline	pipelines/infra.yml or capstone/infra-pipeline.yml	plan in CI; apply gated by Environment approval
App artifact	Still Day 39: promote the image/zip, do not rebuild in staging	Infra apply ≠ app rebuild

Architecture B — apply, approve, destroy

Step	Default	Stop when
plan / what-if	Always in CI; read Modify/Delete	The diff recreates a disk or a vault you did not expect
apply to dev	Auto after green plan on main (or manual once)	You are not on the personal subscription
apply to staging	Environment approval — even if the approver is you	You click-apply from a laptop and skip the pipeline
App deploy	Reuse Day 91/92 artifact against the new outputs	A second compile 'to be safe'
When the screenshot exists	Destroy non-prod (dev first)	Souvenir App Service plans

Architecture C — toy vs portfolio pitfalls

Pitfall	What you see	Fix
Portal-only env	Works on your screen, cannot be rebuilt	Everything in capstone/infra/
Copied folders	infra-dev/ and infra-stg/ drifting	One module, two param files
Dialect tourism	Bicep Monday, Terraform Wednesday	Stay on the Phase 5 tool
Apply without plan	'It is only a SKU'	plan/what-if is mandatory manners
Non-prod souvenirs	Two plans still billing on Day 100	Destroy steps in the README; run them

One-liner to remember

Infra as code for your demo is the difference between a toy and a portfolio piece.

Step-by-step lab (personal Azure / Azure DevOps only)

Same public demo-shipboard as Days 91–92. Personal subscription. Destroy non-prod when the proof exists. No employer modules.

- 1 Add capstone/infra/ in the same dialect you used in Phase 5 (Bicep or Terraform). One module: RG + the runtime (App Service plan/app or Container Apps env) + ACR if still needed.
- 2 Add two parameter files (dev and staging). Different names/SKUs; same module. No copied folder named infra-prod-final-FINAL.
- 3 Write pipelines/infra.yml (or capstone/infra-pipeline.yml): plan/what-if on every run; apply gated by an Azure DevOps Environment.
- 4 Apply dev from the pipeline. Wire Day 91/92 app deploy to the outputs (hostname, ACR). Confirm /health.
- 5 Apply staging only after you read the plan. Promote the same app artifact — do not rebuild.
- 6 Put destroy steps in capstone/README.md. Destroy non-prod when the screenshot exists. Do not keep two plans as souvenirs.
- 7 A stranger should recreate the env from the repo without you on a screenshare. If not, the README is the bug.

Infra pipeline — plan in CI, apply behind an approval

```
trigger:
  paths:
    include: [capstone/infra/**, pipelines/infra.yml]
pool:
  vmImage: ubuntu-latest
stages:
- stage: Plan
  jobs:
  - job: plan
    steps:
  - script: |
      az deployment group what-if -g rg-shipboard-dev \
        -f capstone/infra/main.bicep -p capstone/infra/dev.bicepparam
      displayName: what-if dev (or terraform plan -var-file=dev.tfvars)
- stage: ApplyDev
  dependsOn: Plan
  jobs:
  - deployment: apply
    environment: capstone-dev
    strategy:
      runOnce:
        deploy:
          steps:
  - script: echo Apply only after you read the what-if/plan
    displayName: Gated apply placeholder
```

Done checklist

- capstone/infra/ uses one module and two param files
- Pipeline prints plan/what-if; apply is gated
- Same app artifact promoted onto the IaC outputs
- Non-prod destroy steps are in the README and you ran them (or scheduled tonight)
- No employer modules, no dialect switch for spice

Tomorrow — Day 94

GitHub portfolio setup — open the blinds with a sanitized README a stranger can follow